

Laboratory Notebook 1

Date and time: 8/30 11:30 AM

Participants and contributions: Yewon and Sanjana

Question or objective

What training methods would be the most effective for training a car controller that is both fast and reliable?

What we investigated or changed

We investigated the different training approaches, including Reactive Control and Parameter Optimization, Imitation Learning, Evolutionary Computation and Neuroevolution, Model-Free Reinforcement Learning, Planning and Model-Predictive Control, Hybrid Approaches, and Learned Dynamics or Model-Based Learning. We interacted with AI to determine what may be the best option(s) and why the others fall short of our objective.

Evidence

[ChatGPT discussion](#)

What we observed

We observed that reactive control would be a good place to start, since we can explicitly define the sensors and how it affects the steering and throttle. It can also allow us to understand why some things occurred and diagnose issues earlier on, such as braking too much or turning too sharply. However, since manually determining the most optimal values isn't very scalable, we looked into how CMA-ES could help by automatically evaluating the different parameter combinations based on performance.

Decision and rationale

We decided to try experimenting with reactive control and CMA-ES, with the possibility of combining them into one refined controller. Initially, we would construct a reactive controller that manages the basic driving behaviors like steering according to track geometry and sensor readings, slowing down when there's a curve, accelerating when possible, and maintaining an optimal position on the track. After establishing a baseline performance, we would use CMA-ES to refine the parameters, such as sensor weights, steering gains, throttle levels, corner-speed thresholds, braking distances, and steering smoothing. We agreed that this approach would be the most efficient since it has a good balance between implementation difficulty, computational efficiency, and potential racing performance.

Next steps

1. Familiarize ourselves with the codebase, racing environment, sensors, and controller interface.
2. Determine which sensor readings are most useful for identifying the center of the track, upcoming curves, and safe acceleration opportunities.
3. Implement a stage one reactive controller that can reliably drive around the track.
4. Define quantitative evaluation metrics, including progress/distance traveled, track-completion rate, lap time or average speed, damage/crashes, and consistency across starting-point seeds.
5. Select a fixed collection of seeds so different controllers can initially be compared under equivalent conditions.
6. Record the baseline performance of the manually tuned reactive controller.
7. Learn how CMA-ES works and determine which controller parameters should be optimized.
8. Develop a fitness function that rewards both speed/progress and reliability rather than allowing the optimizer to favor controllers that are extremely fast but frequently crash.
9. Compare the experimental approaches using the same evaluation conditions and use the results to decide what should be carried forward into the refinement stage. 🏁

Laboratory Notebook 2

Date and time: 8/30 8:00 PM

Participants and contributions: Yewon

Question or objective

Develop a minimum viable reactive controller as the baseline for later CMA-ES optimization.

Hypothesis

Local track and sensor observations are enough to determine an appropriate steering direction and throttle level without using a learned model. A simple reactive controller should be able to stay on the track by using nearby sensor information to estimate where the track is going and how sharply the car should turn.

Initial experiment

Implement a basic reactive controller that uses sensor information to keep the car near the center of the track. Steering will be determined from the relative space or track position on the left and right sides of the car. Throttle will depend on steering intensity: the car will use higher throttle on straighter sections and reduce throttle during sharper turns.

The controller will then be evaluated on five fixed random seeds to measure whether it can drive consistently from different starting positions.

What we investigated or changed

We investigated how a reactive controller could use the car's available sensor information to make steering and throttle decisions without machine learning.

We identified two main behaviors for the first controller:

1. Maintaining a reasonable position within the track.
2. Adjusting speed based on how aggressively the car needs to steer.

We also decided to begin with a small number of manually chosen controller parameters, such as steering gain, turn thresholds, and throttle levels. These parameters provide a baseline that can later be optimized automatically using CMA-ES.

Evidence

- **AI-agent assistance:** Implemented and tested a deterministic reactive controller using centerline geometry, lookahead, wall LiDAR, speed, and front-wall safety logic.
- **Commits or code:** Added `src/controllers/reactive_v1.py`. Commit hash: 076b018.
- **Experiment configuration:** Shown in the Experiment Data with controller `reactive_v1`.
- **Experiment output:** Two laps completed on every seed; distances ranged from 386.49 m to 418.40 m; best lap times ranged from 14.45 s to 16.07 s; no eliminations; minor wall contact and damage occurred on all runs.

What we observed

The reactive controller successfully completed every test run, showing that local sensor and track information was sufficient for basic, reliable driving without a learned model. The controller behaved consistently across all five starting seeds and maintained enough control to complete two laps each time.

However, every run included a small amount of wall contact, off-track time, and damage. This suggests that the controller can recover from dangerous situations, but its safety behavior is still mostly reactive. It often corrects after getting too close to a wall rather than anticipating the turn early enough to avoid the situation entirely.

Seed 3 produced the fastest best lap at **14.45 seconds**, while Seed 4 produced the slowest best lap at **16.07 seconds** and the greatest amount of off-track time at **0.600 seconds**. This shows that starting position still has some effect on performance, even though the controller completed every run.

No repeated manual parameter tuning was performed, so these results represent a genuine baseline rather than a heavily optimized controller.

Decision and rationale

We decided to keep the reactive controller as the foundation of our approach because it demonstrated strong reliability across all five seeds. It completed every race, avoided elimination, and consistently defeated the `crash_fast` baseline.

The results support our original hypothesis that local track and sensor observations are sufficient to produce a functional controller without a learned model.

However, the remaining wall contact, off-track time, and variation in lap times indicate that the manually selected parameters are probably not optimal. Because the controller already works reliably, it provides a good foundation for automated parameter optimization.

For this reason, we will keep the reactive controller architecture and later use CMA-ES to search for better values for parameters such as steering gain, throttle levels, turn thresholds, target speed, and wall-avoidance thresholds. This should allow us to improve speed and consistency while preserving the reliability of the current controller.

Next steps

The current controller and its five-seed results will be preserved as the baseline so that future CMA-ES versions can be compared directly against it.

1. Reduce wall contact and off-track time.
2. Reduce unnecessary steering corrections.
3. Improve corner entry and exit speed.
4. Improve lap-time consistency across different seeds.
5. Increase overall race speed without reducing completion reliability.

Experiment Data

Controller: reactive_v1
Seeds: 1, 2, 3, 4, 5

Parameters

Parameter	Value
STEERING_GAIN	0.22
HEADING_GAIN	0.012
EMERGENCY_CORRECTION_GAIN	0.75
STRAIGHT_THROTTLE	0.62
MODERATE_TURN_THROTTLE	0.38
SHARP_TURN_THROTTLE	0.18
MODERATE_TURN_THRESHOLD	0.28
SHARP_TURN_THRESHOLD	0.58
TARGET_SPEED_MPS	5.5
WALL_DANGER_THRESHOLD_M	1.35
WALL_EMERGENCY_THRESHOLD_M	0.72

Results

Seed	Laps	Distance (m)	Best Lap (s)
1	2	403.13	15.45
2	2	399.12	15.43
3	2	418.40	14.45
4	2	386.49	16.07
5	2	398.48	15.40

Overall

- 5/5 completed
- 0 eliminations
- 5/5 wins vs crash_fast
- Small wall contact and minor damage occurred on every run. 🚧

Laboratory Notebook 3

Date and time: 8/31/26
Participants and contributions: Yewon

Question or objective

Can predictive slowing from processed lookahead geometry reduce wall contact and damage while preserving the reliability of reactive_v1?

Hypothesis

The v1 controller reacts primarily to current steering demand and nearby walls. Using the magnitude of future centerline offsets to reduce throttle before a corner should reduce corner-entry speed and collision exposure without changing the steering policy.

Initial experiment

Create reactive_v2 by preserving v1 steering and wall recovery, then add a single behavior: throttle caps based on normalized lookahead offset (abs(offset) / lookahead_distance). Compare v2 directly with v1 under identical deterministic conditions.

What we investigated or changed

We added predictive corner slowing using the public camera.lookahead_offsets_m and camera.lookahead_distances_m observations. No learning, optimization, privileged state, or external dependency was introduced.

Evidence

- **AI-agent assistance:** Implemented reactive_v2, ran static checks, ran the full test suite, and evaluated v1 and v2 on five fixed seeds.
- **Commits or code:** Added src/controllers/reactive_v2.py; exact version is the file contents in this working tree. No commit hash was available.
- **Experiment configuration:** One 30-second race per seed, seeds 1-5, 60 Hz, marshal recovery disabled, team-sum scoring, 1 m win margin, v2 versus v1.

What we observed

Both versions completed all five runs and two laps per run. V2 lost speed and distance on seeds 1, 4, and 5, and won on seeds 2 and 3. Its wall contact remained low, but damage was not consistently lower. The predictive throttle therefore traded away performance without establishing a reliable safety gain.

Decision and rationale

The lookahead-based change is retained as a documented negative experiment, but v1 remains the better baseline. The next optimization should first instrument how often each throttle branch activates and collect per-tick traces before selecting another controller change.

Next steps

Add lightweight branch-activation and sensor-summary logging to the experiment harness, then use that evidence to choose one parameter or behavior for the next controlled experiment.

Overall

- 5/5 v2 runs completed
- 0 eliminations
- 2/5 wins against v1 in the comparison runner
- No measured improvement over v1; v2 was slower overall

Experiment Configuration

	Field	Value	Where verified
Controller version		reactive_v2.py compared with reactive_v1.py	Controller files
Seeds		1, 2, 3, 4, 5	Test commands
Race count		1 per seed	--races 1
Round duration		30.0 seconds	--round-seconds 30 and race output
Timestep		0.016666666666666666 seconds	Race output and GameConfig
Race rules		Team-sum; win margin 1.0 m; marshal disabled	HeadToHeadRaceRules, --no-marshal, race output

Parameters

Parameter	Value	Purpose
LOOKAHEAD_CURVE_THRESHOLD	0.18	Starts moderate predictive slowing
LOOKAHEAD_SHARP_CURVE_THRESHOLD	0.42	Starts sharp predictive slowing
LOOKAHEAD_CURVE_THROTTLE	0.30	Moderate-corner throttle cap
LOOKAHEAD_SHARP_CURVE_THROTTLE	0.16	Sharp-corner throttle cap

All other steering, wall, and speed parameters were kept equal to v1.

Five-seed test results

Version	Seed	Laps	Distance (m)	Best lap (s)	Max speed (m/s)	Damage	Wall contact (s)	Off-track (s)	Result
v2	1	2	394.73	15.70	17.88	0.0098	0.083	0.500	Lost
v2	2	2	392.73	15.90	17.99	0.0096	0.100	0.433	Won
v2	3	2	401.12	15.50	17.94	0.0092	0.083	0.417	Won
v2	4	2	355.85	18.52	17.96	0.0117	0.083	0.350	Lost
v2	5	2	380.53	16.77	17.94	0.0076	0.083	0.400	Lost

The v2 results were not identical to v1. They show lower distance and slower best laps in the direct comparison, while wall contact was equal or lower and off-track time was lower on seeds 2, 4, and 5.

Laboratory Notebook 4

Date and time: 8/31/26

Participants and contributions: Sanjana

Question or objective

Can CMA-ES evolve all weights of a small fixed neural network into a fast and reliable Formula 110 controller that generalizes across starting positions?

Hypothesis

The simulator’s processed camera geometry, wall-only LiDAR, speed, and yaw rate contain enough local information for a small neural network to follow the track. CMA-ES should be able to optimize the complete network without gradients if the model remains small and fitness strongly penalizes inconsistent, damaging, or off-track behavior.

Initial experiment

Implement a fixed 12 → 8 → 2 multilayer perceptron and use CMA-ES to optimize all 122 weights and biases. Evaluate every individual on the same three deterministic training seeds and reserve five different seeds for held-out validation. Use a scalar fitness that rewards forward progress and early lap completion while penalizing off-track time, wall contact, damage, elimination, and inconsistent performance across seeds.

The minimum viable run used 20 individuals for 25 generations. Each training trial lasted 20 simulated seconds at 60 Hz with marshal recovery disabled.

What we investigated or changed

We first inspected the controller API, complete sensor reference, race runtime, progress tracker, headless evaluation utilities, example controllers, and autograder constraints. The controller can use public sensor snapshots but cannot observe official progress or mutable physics state. Official progress, contact, damage, off-track time, lap time, and survival can still be measured externally for fitness.

We selected a compact 12-value observation vector:

1. Signed speed.
2. Yaw rate.
3. Track-center offset.
4. Track-heading error.
5. Camera lookahead offset at 4 m.
6. Camera lookahead offset at 9 m.
7. Camera lookahead offset at 16 m.
8. Wall LiDAR at -90 degrees.
9. Wall LiDAR at -20 degrees.
10. Wall LiDAR at 0 degrees.
11. Wall LiDAR at 20 degrees.
12. Wall LiDAR at 90 degrees.

Absolute world heading, tick, odometry distance, pitch, roll, competitor readings, and contact state were excluded from the initial policy. They were redundant for single-car track following, described failure after it occurred, or could encourage track/time memorization. Infinite LiDAR readings were replaced with a finite cap before normalization.

We implemented dependency-free controller inference and kept `cma` and `NumPy` as training-only dependencies. Each evaluation received a fresh physics world, car, lap tracker, sensor state, and controller. Repeated evaluation of the same weights and seed produced exactly identical metrics.

Evidence

- **AI-agent assistance:** Inspected repository documentation and tests; designed the observation vector, network, individual encoding, fitness, and seed split; implemented the controller and evaluator; added automated tests; executed the full CMA-ES run; and performed isolated Gradescope-style verification.
- **Commits or code:** Added `src/controllers/cmaes_policy.py`, `src/controllers/cmaes_neuroevolution.py`, `src/controllers/cmaes_weights.json`, `src/racing/experiments/neuroevolution.py`, `scripts/train_cmaes.py`, `tests/test_cmaes_neuroevolution.py`, and `docs/CMAES_EXPERIMENT.md`. No commit hash was available during this session.
- **Experiment configuration:** 20 population members, 25 generations, 3 training seeds, 5 held-out validation seeds, 20-second training trials, 60 Hz, CMA-ES optimizer seed 110, initial sigma 0.35, and parameter bounds `[-5, 5]`.
- **Experiment output:** 500 individuals and 1,500 seeded training trials completed in approximately 870 seconds. Raw records were written to `artifacts/cmaes-mvp/metrics.jsonl`; final evaluation was written to `artifacts/cmaes-mvp/final_evaluation.json`.
- **Verification:** All 117 repository tests passed. The new files passed Ruff lint/format checks and strict Pyright checks. The isolated controller worker reproduced the official-seed results.

What we observed

The first populations mostly drove off-track or contacted walls, so early robust fitness values were negative. The best generation-1 fitness was `-1.2253`. Fitness improved substantially during evolution: generation 7 reached `-0.0931`, generation 9 reached `0.2846`, and generation 10 produced the best-ever individual with fitness `1.5571`.

The champion was generation 10, individual 14, with parameter checksum `ef6aa46e014b22bd`. Later generations did not surpass this individual, but the population distribution continued improving. The generation mean increased from approximately `-1.87` at the beginning to positive values in the final part of the run. Held-out validation fitness improved from `-1.3597` at generation 5 to `1.5138` at generation 10.

In independent 30-second evaluation before any later speed adjustment, all three training seeds, all five held-out seeds, and both official seeds completed two laps with zero damage, zero wall contact, and zero off-track time. Held-out best laps were approximately 10.95 to 11.02 seconds. The results were almost identical across seed groups, providing evidence that the controller learned a general local track-following policy rather than memorizing one start.

Decision and rationale

The minimum viable CMA-ES experiment succeeded and justified continuing with neuroevolution. It produced a controller that was faster and safer than the manually written reactive baseline while using only public observations. On official seeds, the evolved controller’s best laps were approximately 11.0 seconds compared with approximately 12.4 seconds for reactive_v1, and the evolved controller had zero damage and wall contact.

We retained the generation-10 champion rather than the final generation’s best individual because checkpointing tracked the best result across the entire run. We also retained the small architecture because 122 parameters were sufficient; increasing network size was not supported by current evidence.

Next steps

- 1. Test whether a small increase in throttle can improve distance and lap time without introducing damage or wall contact.
- 2. Change only one output parameter first so the effect can be attributed clearly.
- 3. Evaluate every candidate adjustment on the same held-out and official seeds.
- 4. Reject aggressive settings that gain speed by crashing, contacting walls, or failing from some starts.
- 5. Preserve the original evolved weights so the safety/speed tradeoff remains reproducible.

Overall

- 25 generations and 500 CMA-ES individuals evaluated
- 1,500 deterministic training trials completed
- 10/10 independent 30-second runs completed across training, validation, and official seeds
- 2 laps completed on every independent run
- 0 eliminations, 0 damage, 0 wall contact, and 0 off-track time
- Best official lap approximately 10.97 seconds
- CMA-ES was successful enough to continue as the primary refinement approach

Experiment Configuration

Field	Value	Purpose
Architecture	12 -> 8 -> 2	Small fixed MLP suitable for black-box optimization
Evolved parameters	122	All network weights and biases
Population	20	Candidate policies per generation
Generations	25	Minimum viable optimization budget
Training seeds	17, 83, 241	Optimize across multiple starting positions
Validation seeds	41, 137, 311, 509, 887	Measure generalization without updating CMA-ES
Official test seeds	110, 2026	Final Gradescope-style verification only
Training trial duration	20 seconds	Balance behavioral evidence and computation
Final trial duration	30 seconds	Match official evaluation
Timestep	1/60 second	Standard simulator control rate
Initial sigma	0.35	Initial CMA-ES search spread
Parameter bounds	[-5, 5]	Prevent extreme network parameters
Optimizer seed	110	Make the evolutionary run reproducible

Parameters

Parameter	Value	Purpose
INPUT_SIZE	12	Compact normalized sensor vector
HIDDEN_SIZE	8	Small nonlinear representation
OUTPUT_SIZE	2	Steering and throttle
PARAMETER_COUNT	122	CMA-ES search-vector length
SPEED_CAP_MPS	12.0	Speed normalization cap
YAW_RATE_CAP_DEGREES_PER_S	180.0	Yaw-rate normalization cap
CENTER_OFFSET_CAP_M	6.0	Center-offset normalization cap
LOOKAHEAD_OFFSET_CAP_M	12.0	Lookahead normalization cap
LIDAR_CAP_M	20.0	Finite replacement and range cap
MIN_THROTTLE	0.05	Prevent completely stationary random policies
Wall LiDAR angles	(-90, -20, 0, 20, 90)	Side and forward wall clearance
Hidden activation	tanh	Bounded signed hidden representation
Steering activation	tanh	Produces steering in [-1, 1]
Throttle activation	sigmoid	Produces forward throttle in [0.05, 1]

Five-seed held-out validation results

These results are from the original CMA-ES champion before the Session 5 throttle-bias refinement.

Seed	Laps	Distance (m)	Best lap (s)	Max speed (m/s)	Damage	Wall contact (s)	Off-track (s)
------	------	--------------	--------------	-----------------	--------	------------------	---------------

41	2	424.07	11.00	16.93	0.0000	0.000	0.000
137	2	408.84	10.95	17.19	0.0000	0.000	0.000
311	2	413.23	11.02	17.22	0.0000	0.000	0.000
509	2	425.43	11.00	16.90	0.0000	0.000	0.000
887	2	410.29	10.97	17.11	0.0000	0.000	0.000

Laboratory Notebook 5

Date and time: 8/31/26

Participants and contributions: Sanjana

Question or objective

Can the CMA-ES champion be made faster with a small, controlled output adjustment while preserving its zero-damage and zero-wall-contact reliability?

Hypothesis

The original evolved controller had best laps near 11 seconds and zero contact, suggesting that it may have retained a small safety margin. Increasing only the throttle output bias should improve acceleration and progress without changing the learned steering policy. Small increases may preserve reliability, while larger increases are expected to produce off-track driving, contact, and damage.

Initial experiment

Keep all 122 evolved weights fixed except the final throttle-output bias. Evaluate several positive offsets under identical 30-second conditions on five held-out validation seeds and the two official seeds. Compare mean distance, best-lap time, maximum speed, damage, wall contact, off-track time, and minimum completed laps.

The first sweep tested offsets of 0.00, 0.15, 0.30, 0.50, and 0.75. A narrower follow-up sweep tested 0.04, 0.07, 0.10, 0.12, and 0.14 to make the speed/safety boundary explicit.

What we investigated or changed

We first compared the original CMA-ES champion with reactive_v1 on official seeds 110 and 2026. The reactive controller reached a higher momentary top speed of approximately 18.0 m/s, but its best laps were slower at approximately 12.4 seconds and it accumulated wall contact and damage. The CMA-ES policy had a lower top speed of approximately 17.2 m/s but completed best laps near 11.0 seconds with no damage or contact. This showed that lap speed, not momentary maximum speed, was the correct performance target.

We then changed only the final neural-network output bias controlling throttle. Steering weights, sensor preprocessing, network architecture, fitness, and all other evolved parameters remained fixed. The selected adjustment was:

POST_TRAINING_THROTTLE_BIAS_OFFSET = +0.15

The final stored throttle-output bias changed from approximately -1.38972 to -1.23972. The change was recorded in cmaes_weights.json metadata.

Evidence

- **AI-agent assistance:** Benchmarked the original controller against reactive_v1; ran coarse and fine throttle-bias sweeps; compared distance, lap time, maximum speed, damage, contact, and off-track time; updated the saved artifact; and reran held-out and isolated official evaluations.
- **Commits or code:** Updated src/controllers/cmaes_weights.json. The learned steering policy and controller source code were unchanged. No commit hash was available during this session.
- **Experiment configuration:** Seven evaluation seeds per candidate adjustment: held-out seeds 41, 137, 311, 509, and 887, plus official seeds 110 and 2026. Each trial lasted 30 seconds at 60 Hz with marshal recovery disabled.
- **Experiment output:** The +0.15 setting increased mean distance from 417.05 m to 436.95 m and reduced mean best-lap time from 10.986 seconds to 10.605 seconds across seven runs.
- **Verification:** The final controller completed every training, validation, and official run with two laps, zero damage, zero wall contact, and no elimination. All 117 repository tests passed; new code continued to pass Ruff and strict Pyright checks.

What we observed

Increasing throttle produced a clear speed/reliability tradeoff:

Throttle-bias offset	Mean distance (m)	Mean best lap (s)	Maximum speed (m/s)	Total damage	Wall contact (s)	Off-track (s)	Minimum laps
0.00	417.05	10.986	17.25	0.000	0.000	0.000	2
0.15	436.95	10.605	18.10	0.000	0.000	1.900	2
0.30	433.29	11.071	18.99	0.330	1.017	3.750	2
0.50	373.35	10.343*	20.29	1.325	29.183	31.500	0

0.75	252.16	6.881*	22.06	3.161	85.650	88.183	0
------	--------	--------	-------	-------	--------	--------	---

* The apparently fast lap averages at aggressive offsets are misleading because some runs failed to complete a lap. These settings had substantial collision and off-track penalties and were not reliable controllers.

The +0.15 setting improved mean 30-second progress by approximately 4.8%, reduced mean best-lap time by approximately 0.38 seconds, and increased top speed by about 0.85 m/s. It preserved zero damage and zero wall contact across all seven development/test runs. Its cost was 1.9 seconds of total off-track time across 210 simulated seconds, less than 1% of evaluation time.

The narrow sweep showed that even a +0.04 increase introduced 0.117 seconds of total off-track time. Therefore, there was no tested positive throttle adjustment that was both meaningfully faster and retained exactly zero off-track time. The choice was a deliberate tradeoff rather than a free improvement.

Decision and rationale

We selected the +0.15 throttle-output bias because it provided the largest reliable improvement before damage and wall contact appeared. The +0.30 and larger settings were rejected because they converted extra speed into collisions, damage, and inconsistent completion. The +0.04 through +0.14 settings provided smaller speed improvements but still introduced some off-track time.

The final controller favors faster leaderboard-relevant progress while preserving the most important safety requirements: survival, zero damage, and zero wall contact. We documented the small off-track tradeoff explicitly rather than describing the adjustment as an unconditional improvement.

Next steps

- 1. Use another CMA-ES refinement run centered on the champion to learn steering changes that support the higher throttle without leaving the track.
- 2. Keep the +0.15 controller and original zero-off-track controller as separate checkpoints for controlled comparison.
- 3. Evaluate future candidates on additional unseen seeds rather than repeatedly tuning against the five current validation seeds.
- 4. Test head-to-head performance against human control and the reactive baseline.
- 5. Consider adding a stronger off-track penalty during fine-tuning if exact track containment is more important than the current speed gain.

Overall

- 10/10 final runs completed across training, validation, and official seeds
- 2 laps completed on every run
- 0 eliminations, 0 damage, and 0 wall contact
- Mean seven-seed progress improved by approximately 4.8%
- Mean seven-seed best lap improved from 10.986 to 10.605 seconds
- Official-seed progress increased from 422.35/415.12 m to 441.62/434.85 m
- Small documented off-track tradeoff: less than 1% of total seven-seed evaluation time

Experiment Configuration

Field	Value	Purpose
Base controller	Generation-10 CMA-ES champion	Preserve learned steering and sensor behavior
Changed parameter	Final throttle-output bias only	Isolate the cause of performance change
Coarse offsets	0.00, 0.15, 0.30, 0.50, 0.75	Identify the useful and unsafe regions
Fine offsets	0.04, 0.07, 0.10, 0.12, 0.14	Examine the speed/off-track boundary
Selected offset	+0.15	Best tested gain before damage/contact appeared
Evaluation seeds	41, 137, 311, 509, 887, 110, 2026	Held-out and official positions
Final additional seeds	17, 83, 241	Confirm behavior on training positions
Trial duration	30 seconds	Match official evaluation
Timestep	1/60 second	Standard simulator control rate
Marshal recovery	Disabled	Prevent automatic reset from hiding failure

Parameters

Parameter	Before	After	Purpose
Post-training throttle-bias offset	0.00	+0.15	Increase acceleration and sustained speed
Stored throttle-output bias	-1.38972	-1.23972	Neural-network throttle logit bias
Network architecture	12 -> 8 -> 2	unchanged	Preserve the learned policy
Other evolved parameters	121 values	unchanged	Make this a one-variable experiment

Five-seed held-out validation results

Seed	Laps	Distance (m)	Best lap (s)	Damage	Wall contact (s)	Off-track (s)
41	2	444.71	10.62	0.0000	0.000	0.250
137	2	427.22	10.55	0.0000	0.000	0.317

311	2	434.38	10.65	0.0000	0.000	0.167
509	2	446.15	10.63	0.0000	0.000	0.250
887	2	429.75	10.58	0.0000	0.000	0.283

Official-seed results

Seed	Laps	Distance (m)	Best lap (s)	Max speed (m/s)	Damage	Wall contact (s)
110	2	441.62	10.62	18.10	0.0000	0.000
2026	2	434.85	10.58	18.00	0.0000	0.000